

Day 21: Destructuring in JavaScript

Learning Objectives

- Understand what destructuring means.
- Destructure arrays.
- Destructure objects.
- Rename destructured variables.
- Use default values.
- Destructure nested objects.
- Understand why destructuring is important in React.

1. Why Are We Learning Destructuring?

Students already know:

```
const student = {  
  name: "Arun",  
  age: 21,  
  course: "React"  
};
```

Normally, to access the values:

```
console.log(student.name);  
console.log(student.age);  
console.log(student.course);
```

This works, but when working with React, we frequently need several properties from an object.

Destructuring gives us a cleaner way:

```
const { name, age, course } = student;  
  
console.log(name);  
console.log(age);
```

```
console.log(course);
```

Simple definition

Destructuring is a JavaScript feature that allows us to extract values from arrays or properties from objects and store them in variables.

2. Destructuring an Object

Start with:

```
const student = {  
  name: "Arun",  
  age: 21,  
  course: "React"  
};
```

Without destructuring:

```
const name = student.name;  
const age = student.age;  
const course = student.course;
```

With destructuring:

```
const { name, age, course } = student;
```

Now:

```
console.log(name);  
console.log(age);  
console.log(course);
```

Output:

```
Arun  
21  
React
```

3. How Does It Work?

Visually

student

```
|  
├── name  → "Arun"  
├── age   → 21  
└── course → "React"
```

↓

```
const { name, age, course } = student;
```

↓

```
name = "Arun"  
age  = 21  
course = "React"
```

The property names on the left match the property names in the object.

4. Order Does NOT Matter with Objects

This is important.

```
const student = {  
  name: "Arun",  
  age: 21,  
  course: "React"  
};
```

```
const { course, name, age } = student;
```

This still works.

Why?

Because object destructuring works using property names, not position.

5. Array Destructuring

Now compare with an array:

```
const colors = ["red", "green", "blue"];
```

Without destructuring:

```
const first = colors[0];  
const second = colors[1];  
const third = colors[2];
```

With destructuring:

```
const [first, second, third] = colors;
```

Now:

```
console.log(first);  
console.log(second);  
console.log(third);
```

Output:

```
red  
green  
blue
```

6. Important Difference — Object vs Array

Object

Uses:

```
const { name, age } = student;
```

Uses property names.

Array

Uses:

```
const [first, second] = colors;
```

Uses position/order.

Put this on the board:

OBJECT

```
{ name, age }
```

↓

property name

ARRAY

```
[ first, second ]
```

↓

position

7. Skipping Array Values

Suppose:

```
const colors = ["red", "green", "blue"];
```

We only want "blue".

```
const [, , third] = colors;
```

```
console.log(third);
```

Output:

blue

The commas mean:

Position 0 → skip

Position 1 → skip

Position 2 → take

This is useful, but don't overemphasize it at beginner level.

8. Renaming Object Properties

Suppose:

```
const student = {  
  name: "Arun",  
  age: 21  
};
```

We want the variable to be called studentName.

```
const { name: studentName, age: studentAge } = student;
```

Now:

```
console.log(studentName);  
console.log(studentAge);
```

Output:

Arun

21

Understand the syntax

```
const { name: studentName } = student;
```

Means:

Object property → Variable name

name → studentName

9. Default Values

What happens if the property doesn't exist?

```
const student = {  
  name: "Arun"  
};
```

```
const { name, course } = student;
```

```
console.log(course);
```

Output:

undefined

We can provide a default:

```
const { name, course = "Not Assigned" } = student;
```

```
console.log(course);
```

Output:

Not Assigned

Important

The default is used only when the value is undefined.

10. Combining Rename + Default

```
const student = {  
  name: "Arun"  
};
```

```
const {  
  name: studentName,  
  course: studentCourse = "Not Assigned"  
} = student;
```

Now:

```
console.log(studentName);  
console.log(studentCourse);
```

Output:

```
Arun  
Not Assigned
```

11. Nested Object Destructuring

Consider:

```
const student = {  
  name: "Arun",  
  age: 21,  
  address: {  
    city: "Puducherry",
```

```
    pincode: "605001"
  }
};
```

We can extract:

```
const {
  name,
  address: { city, pincode }
} = student;
```

Now:

```
console.log(name);
console.log(city);
console.log(pincode);
```

Output:

```
Arun
Puducherry
605001
```

For beginners, explain it as:

```
student
├── name
├── age
└── address
    ├── city
    └── pincode
```

12. Destructuring Function Parameters

This is especially important for React.

Suppose:

```
const displayStudent = (student) => {  
  console.log(student.name);  
  console.log(student.course);  
};
```

We can destructure directly in the function parameter:

```
const displayStudent = ({ name, course }) => {  
  console.log(name);  
  console.log(course);  
};
```

Call:

```
displayStudent({  
  name: "Arun",  
  course: "React"  
});
```

Output:

Arun
React

Why is this important?

Students will see this pattern frequently in React:

```
const Student = ({ name, course }) => {  
  // ...  
};
```

This is one of the most important reasons to teach destructuring before React.

13. Array Destructuring in Function Parameters

```
const displayColors = ([first, second, third]) => {  
  console.log(first);  
  console.log(second);  
  console.log(third);  
};
```

```
displayColors(["Red", "Green", "Blue"]);
```

Output:

```
Red  
Green  
Blue
```

14. Destructuring with map()

Suppose:

```
const students = [  
  {  
    name: "Arun",  
    course: "React"  
  },  
  {  
    name: "Priya",  
    course: "Python"  
  },  
  {  
    name: "Kumar",  
    course: "JavaScript"  
  }  
];
```

Without destructuring:

```
students.forEach((student) => {
```

```
    console.log(student.name);  
    console.log(student.course);  
  });
```

With destructuring:

```
students.forEach(({ name, course }) => {  
  console.log(name);  
  console.log(course);  
});
```

This is a very useful React pattern.

15. Practical Example — Product List

```
const products = [  
  {  
    name: "Laptop",  
    price: 50000  
  },  
  {  
    name: "Mouse",  
    price: 1000  
  },  
  {  
    name: "Keyboard",  
    price: 2000  
  }  
];
```

Using destructuring:

```
products.forEach(({ name, price }) => {  
  console.log(`${name} - ₹${price}`);  
});
```

Output:

Laptop - ₹50000

Mouse - ₹1000

Keyboard - ₹2000

Students can immediately see the advantage.

16. Destructuring + map()

```
const productNames = products.map(({ name }) => {  
  return name;  
});
```

Output:

```
[  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
]
```

Using the arrow-function style students have been practicing:

```
const productNames = products.map(({ name }) => name);
```

This is exactly the type of syntax students will frequently encounter in React.

17. Real React Connection

This is where you should tell students:

Destructuring is not a React feature. It is JavaScript. React uses it heavily.

For example, React component props:

```
const Student = ({ name, age, course }) => {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>{age}</p>  
      <p>{course}</p>  
    </div>  
  );  
};
```

Here:

```
({ name, age, course })
```

is simply object destructuring.

Students already learned it in JavaScript.

18. Another React Connection — useState

Later in React they will see:

```
const [count, setCount] = useState(0);
```

This is array destructuring.

Conceptually:

```
const values = [0, someFunction];
```

```
const [count, setCount] = values;
```

So:

Array Destructuring



React useState()

This is a very important connection to show students.

Common Beginner Mistakes

Mistake 1 — Wrong object property

```
const student = {  
  name: "Arun"  
};
```

```
const { studentName } = student;
```

This doesn't automatically rename name.

Correct:

```
const { name: studentName } = student;
```

Mistake 2 — Confusing object and array syntax

Object:

```
const { name } = student;
```

Array:

```
const [first] = students;
```

Remember:

Object → {}

Array → []

Mistake 3 — Thinking order matters for objects

```
const { course, name, age } = student;
```

is perfectly valid.

For arrays:

```
const [first, second] = courses;
```

Position matters.

Summary

Object Destructuring

```
const { name, age } = student;
```

Array Destructuring

```
const [first, second] = courses;
```

Rename

```
const { name: studentName } = student;
```

Default Value

```
const { course = "Not Assigned" } = student;
```

Function Parameter Destructuring

```
const displayStudent = ({ name, course }) => {  
  console.log(name, course);  
};
```

Array of Objects

```
students.forEach(({ name, course }) => {  
  console.log(name, course);  
});
```

— Final Concept Map

